

Modelowanie i prognozowanie cen gazu TTF

Analiza ekonometryczna wielowymiarowa: ARIMAX + GARCH-X + kointegracja

Maciek Gilecki, Wioletta Grabias

24.05.2026

Spis treści

1	Konfiguracja środowiska	2
1.1	Instalacja i wczytanie pakietów	2
1.2	Parametry projektu	3
2	Wprowadzenie	3
2.1	Kontekst ekonomiczny	3
2.2	Mechanizm rynku — analiza popytu i podaży	4
2.3	Cel projektu	4
2.4	Hipotezy badawcze	4
3	Pobranie i przygotowanie danych	5
3.1	Źródła danych	5
3.2	Brent i EUR/USD (FRED)	5
3.3	Pogoda i HDD (Open-Meteo)	6
3.4	Zapasy gazu UE (AGSI+ — ręczny eksport CSV)	7
3.5	TTF (Kaggle / Investing.com)	9
3.6	Scalenie i raport jakości	9
3.7	Statystyki opisowe	10
4	Analiza eksploracyjna (EDA)	11
4.1	Wykresy szeregów	11
4.2	Log-stopowy zwrotu i pierwsze różnice	13
4.3	Korelacje	14
5	Testy stacjonarności	15

6	Analiza kointegracji	16
6.1	Engle-Granger (TTF vs Brent)	16
6.2	Johansen (TTF, Brent, EUR/USD)	17
7	Identyfikacja ARIMA	18
8	Estymacja ARIMA i ARIMAX	18
8.1	Model bazowy: ARIMA jednowymiarowy	18
8.2	Model rozszerzony: ARIMAX	19
8.3	Porównanie ARIMA vs ARIMAX	20
8.4	Diagnostyka reszt ARIMAX	20
9	Test efektu ARCH	22
10	Estymacja modelu GARCH-X	23
10.1	sGARCH(1,1) bazowy (bez X)	23
10.2	sGARCH(1,1)-X — pełny	26
10.3	Diagnostyka GARCH-X	28
10.4	Porównanie wariantów GARCH	29
11	Walidacja out-of-sample	30
11.1	Walk-forward dla ARIMA / ARIMAX	31
11.2	Test Diebolda-Mariano	32
11.3	Wizualizacja prognoz OOS	32
12	Prognoza przyszła	33
12.1	ARIMAX ($h = 30$ dni)	33
12.2	GARCH-X (zmiennosc warunkowa)	34
13	Podsumowanie i wnioski	36
13.1	Weryfikacja hipotez	36
13.2	Wnioski kluczowe	36
13.3	Ograniczenia i kierunki rozszerzeń	37

1 Konfiguracja środowiska

1.1 Instalacja i wczytanie pakietów

```

required_packages <- c(
  # Manipulacja danymi
  "tidyverse", "lubridate", "zoo", "readr",
  # Pobieranie i I/O
  "quantmod", "httr", "jsonlite",
  # Modelowanie ekonometryczne
  "forecast", "tseries", "FinTS", "rugarch", "urca",
  # Statystyka, wizualizacja, raporty
  "moments", "xts", "patchwork", "GGally",
  "knitr", "kableExtra"
)

to_install <- setdiff(required_packages, rownames(installed.packages()))
if (length(to_install) > 0) {
  install.packages(to_install, repos = "https://cloud.r-project.org")
}
invisible(lapply(required_packages, library, character.only = TRUE))

theme_set(theme_minimal(base_size = 11))

```

1.2 Parametry projektu

```

START_DATE <- as.Date("2015-01-01")
END_DATE <- as.Date("2024-11-29")

RAW_DIR <- file.path("data", "raw")
PROC_DIR <- file.path("data", "processed")
dir.create(RAW_DIR, showWarnings = FALSE, recursive = TRUE)
dir.create(PROC_DIR, showWarnings = FALSE, recursive = TRUE)

cat("Zakres analizy:", as.character(START_DATE), "-", as.character(END_DATE), "\n")

```

```
## Zakres analizy: 2015-01-01 - 2024-11-29
```

2 Wprowadzenie

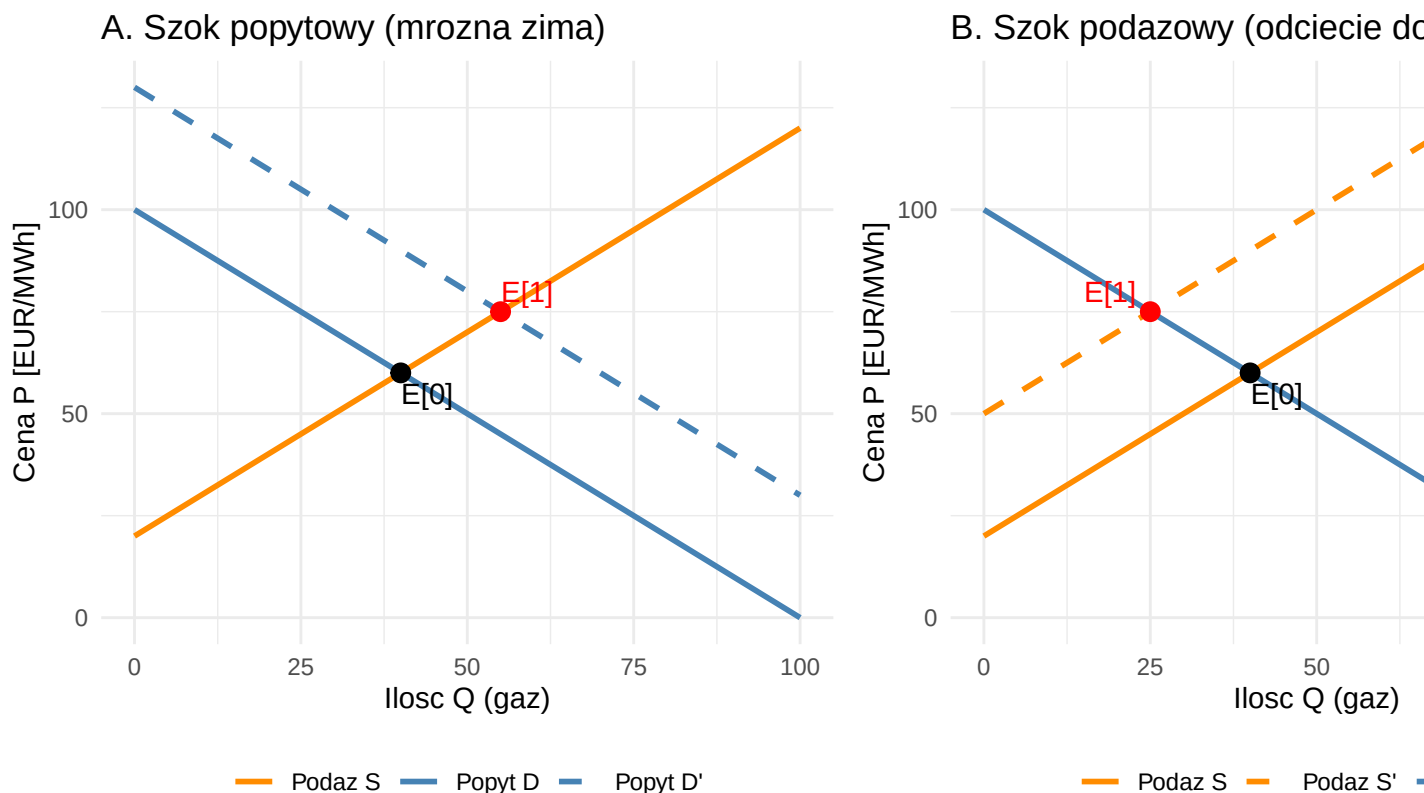
2.1 Kontekst ekonomiczny

Title Transfer Facility (TTF) to holenderski wirtualny hub gazowy — główny benchmark cen gazu ziemnego w Europie. Notowania wyrażane są w EUR/MWh i odzwierciedlają równowagę popytu i podaży na rynku europejskim. Dynamika cen TTF jest determinowana przez czynniki popytowe (temperatura, aktywność przemysłowa), podażowe (zapasy magazynowe, dostawy rurociągowe, import LNG), makroekonomiczne (kurs EUR/USD, ceny ropy Brent) oraz geopolityczne (konflikty, sankcje, decyzje regulacyjne).

Szczególnym przypadkiem był rok 2022, gdy odcięcie dostaw z Rosji doprowadziło do wzrostu cen TTF do ponad 300 EUR/MWh (sierpień 2022) i dramatycznego wzrostu zmienności szeregu czasowego.

2.2 Mechanizm rynku — analiza popytu i podaży

Klasyczna analiza mikroekonomiczna pokazuje, jak szoki popytowe i podażowe wpływają na cenę równowagi rynkowej.



Rysunek 1: Wpływ szoków popytowych i podażowych na rynek gazu TTF

Panel A: mroźna zima przesunęła krzywą popytu w prawo — wyższa cena, wyższy wolumen. **Panel B:** odcięcie dostaw 2022 przesunęło krzywą podaży w lewo — wyższa cena, niższy wolumen.

2.3 Cel projektu

Zbudowanie wielowymiarowego modelu ekonometrycznego prognozującego cenę i zmienność gazu TTF, w oparciu o regresory fundamentalne: cenę ropy Brent, kurs EUR/USD, poziom zapasów magazynowych UE oraz indeks Heating Degree Days (HDD).

2.4 Hipotezy badawcze

- **H1:** Szereg cen TTF jest niestacjonarny ($I(1)$) i wymaga różnicowania.
- **H2:** Między cenami TTF, Brent i EUR/USD istnieje długookresowa relacja kointegrująca.
- **H3:** W resztach modelu ARIMAX występuje istotny efekt ARCH.
- **H4:** Model GARCH(1,1)-X z poziomem zapasów w równaniu wariancji wyjaśnia heteroskedastyczność lepiej niż klasyczny sGARCH(1,1).

- **H5:** Włączenie regresorów fundamentalnych istotnie poprawia jakość prognozy out-of-sample względem modelu jednowymiarowego (test Diebolda-Mariano).

3 Pobranie i przygotowanie danych

3.1 Źródła danych

Zmienna	Opis	Źródło	Plik
TTF	Cena front-month TTF [EUR/MWh]	Investing.com / Kaggle	data/raw/ttf.csv
Brent	Ropa Brent [USD/bbl]	FRED DCOILBRETEU	pobierane online
EURUSD	USD za 1 EUR	FRED DEXUSEU	pobierane online
Storage_pct	% wypełnienia magazynów UE	AGSI+ (eksport CSV ze strony)	data/raw/storage_eu.csv
HDD	Heating Degree Days (Europa Zach.)	Open-Meteo Historical	pobierane online

Wszystkie szeregi pobierane online są cache'owane do data/raw/ — drugi knit nie powtarza pobierania.

3.2 Brent i EUR/USD (FRED)

```
fred_path_brent <- file.path(RAW_DIR, "brent.csv")
fred_path_eurusd <- file.path(RAW_DIR, "eurusd.csv")

# --- Brent ---
if (!file.exists(fred_path_brent)) {
  download.file(
    "https://fred.stlouisfed.org/graph/fredgraph.csv?id=DCOILBRETEU",
    destfile = fred_path_brent, mode = "wb", quiet = TRUE
  )
}
brent <- read_csv(fred_path_brent, show_col_types = FALSE) |>
  rename(Date = 1, Brent = 2) |>
  mutate(Date = as.Date(Date), Brent = as.numeric(Brent)) |>
  filter(!is.na(Brent), Date >= START_DATE, Date <= END_DATE)

# --- EUR/USD ---
if (!file.exists(fred_path_eurusd)) {
  download.file(
    "https://fred.stlouisfed.org/graph/fredgraph.csv?id=DEXUSEU",
    destfile = fred_path_eurusd, mode = "wb", quiet = TRUE
  )
}
eurusd <- read_csv(fred_path_eurusd, show_col_types = FALSE) |>
  rename(Date = 1, EURUSD = 2) |>
  mutate(Date = as.Date(Date), EURUSD = as.numeric(EURUSD)) |>
  filter(!is.na(EURUSD), Date >= START_DATE, Date <= END_DATE)

cat("Brent :", nrow(brent), "obs.\n")
```

```
## Brent : 2520 obs.
```

```
cat("EURUSD :", nrow(eurusd), "obs.\n")
```

```
## EURUSD : 2477 obs.
```

3.3 Pogoda i HDD (Open-Meteo)

Heating Degree Days liczone jako $\max(18 - T_{mean}, 0)$, ważone udziałem poszczególnych aglomeracji w europejskim popycie gazowym.

```
weather_path <- file.path(RAW_DIR, "weather_hdd.csv")

EU_CITIES <- tibble(
  city = c("Frankfurt", "Paris", "Amsterdam", "London", "Brussels", "Milan"),
  lat = c(50.11, 48.85, 52.37, 51.51, 50.85, 45.46),
  lon = c( 8.68, 2.35, 4.90, -0.13, 4.35, 9.19),
  weight = c(0.35, 0.20, 0.10, 0.20, 0.05, 0.10)
)

fetch_year_chunk <- function(lat, lon, year_start, year_end, max_tries = 3) {
  last_err <- NULL
  for (attempt in seq_len(max_tries)) {
    resp <- tryCatch(
      GET(
        "https://archive-api.open-meteo.com/v1/archive",
        query = list(
          latitude = lat, longitude = lon,
          start_date = year_start, end_date = year_end,
          daily = "temperature_2m_mean",
          timezone = "Europe/Berlin"
        ),
        config(connecttimeout = 30),
        httr::timeout(180)
      ),
      error = function(e) { last_err <- e; NULL }
    )
    if (!is.null(resp) && status_code(resp) == 200) {
      j <- fromJSON(content(resp, "text", encoding = "UTF-8"))
      return(tibble(
        Date = as.Date(j$daily$time),
        tmean = as.numeric(j$daily$temperature_2m_mean)
      ))
    }
    Sys.sleep(2 * attempt)
  }
  stop(if (!is.null(last_err)) conditionMessage(last_err) else "HTTP error")
}

fetch_city_chunked <- function(city, lat, lon, weight) {
  years <- seq(year(START_DATE), year(END_DATE))
  cat(" ", city, "(", length(years), "lat) ")
}
```

```

parts <- list()
for (yr in years) {
  yr_start <- max(as.Date(paste0(yr, "-01-01")), START_DATE)
  yr_end <- min(as.Date(paste0(yr, "-12-31")), END_DATE)
  parts[[as.character(yr)]] <- fetch_year_chunk(lat, lon,
                                              format(yr_start, "%Y-%m-%d"),
                                              format(yr_end, "%Y-%m-%d"))

  cat(".")
}
cat(" OK\n")
bind_rows(parts) |> mutate(weight = weight, city = city)
}

if (!file.exists(weather_path)) {
  weather <- tryCatch({
    rows <- purrr::pmap_dfr(EU_CITIES, fetch_city_chunked)
    rows |>
      filter(!is.na(tmean)) |>
      group_by(Date) |>
      summarise(T_mean_EU = sum(tmean * weight) / sum(weight), .groups = "drop") |>
      mutate(HDD = pmax(18 - T_mean_EU, 0)) |>
      arrange(Date)
  },
  error = function(e) {
    message("UWAGA: pobieranie pogody nie powiodlo sie: ", conditionMessage(e))
    message("Kontynuujecie BEZ HDD - model bedzie mial 4 zmienne zamiast 5.")
    NULL
  })

  if (!is.null(weather)) write_csv(weather, weather_path)
} else {
  weather <- read_csv(weather_path, show_col_types = FALSE)
}

```

```
## Frankfurt ( 10 lat)
```

```

if (!is.null(weather)) {
  cat("Pogoda :", nrow(weather), "obs.\n")
} else {
  cat("Pogoda : POMINIETA (problem z Open-Meteo)\n")
}

```

```
## Pogoda : POMINIETA (problem z Open-Meteo)
```

3.4 Zapasy gazu UE (AGSI+ — ręczny eksport CSV)

Plik `data/raw/storage_eu.csv` ściągany ze strony <https://agsi.gie.eu/data-overview/EU> (przycisk *Download CSV*). Parser auto-wykrywa separator i nazwy kolumn.

```

# Szukamy pliku po wzorcach: storage_eu.csv, agsi*.csv, StorageData_GIE_*.csv
storage_candidates <- c(

```

```

list.files(RAW_DIR, pattern = "^storage.*\\.csv$", full.names = TRUE, ignore.case = TRUE),
list.files(RAW_DIR, pattern = "^agsi.*\\.csv$", full.names = TRUE, ignore.case = TRUE),
list.files(RAW_DIR, pattern = "^StorageData.*\\.csv$", full.names = TRUE, ignore.case = TRUE)
)
storage_path <- storage_candidates[1]

if (!is.na(storage_path) && file.exists(storage_path)) {
  first_line <- readLines(storage_path, n = 1, warn = FALSE)
  sep <- if (length(gregexpr(";", first_line)[[1]]) >
    length(gregexpr(",", first_line)[[1]])) ";" else ","

  raw_storage <- read.csv(storage_path, sep = sep, stringsAsFactors = FALSE,
    check.names = FALSE, na.strings = c("", "NA", "-"))

  # "Gas Day Start (...)", "Gas Day", "gasDayStart", "Date"...
  date_col <- names(raw_storage)[grepl("gas\\s*day|^date$|gasdaystart",
    names(raw_storage), ignore.case = TRUE)][1]

  # "Full (%)", "Full", "Storage_pct"...
  full_col <- names(raw_storage)[grepl("full", names(raw_storage),
    ignore.case = TRUE)][1]

  if (is.na(date_col) || is.na(full_col)) {
    warning("Nie rozpoznano kolumn w ", basename(storage_path),
      " - dostępne: ", paste(names(raw_storage), collapse = ", "))
    storage <- NULL
  } else {
    storage <- tibble(
      Date = as.Date(parse_date_time(raw_storage[[date_col]],
        orders = c("ymd", "dmy", "mdy"))),
      Storage_pct = as.numeric(gsub(",", ".", as.character(raw_storage[[full_col]])))
    ) |>
      filter(!is.na(Date), !is.na(Storage_pct)) |>
      arrange(Date) |>
      distinct(Date, .keep_all = TRUE)

    span_days <- as.numeric(max(storage$Date) - min(storage$Date))
    cat("Storage:", nrow(storage), "obs. od", as.character(min(storage$Date)),
      "do", as.character(max(storage$Date)),
      "(plik:", basename(storage_path), ")\n")

    if (span_days < 365) {
      warning("Plik storage pokrywa tylko ", span_days, " dni - za malo dla modelu.\n",
        "Wroc na https://agsi.gie.eu/data-overview/EU, ustaw zakres dat\n",
        "od 2015-01-01 do 2024-11-29 i pobierz ponownie.")
    }
  }
} else {
  storage <- NULL
  warning("Brak pliku storage w ", RAW_DIR,
    " - pobierz z https://agsi.gie.eu/data-overview/EU.")
}

```

```
## Storage: 3616 obs. od 2015-01-01 do 2024-11-24 (plik: StorageData_GIE_2015-01-01_2024-11-24.csv )
```


3.5 TTF (Kaggle / Investing.com)

```
ttf <- read_csv(file.path(RAW_DIR, "ttf.csv"), show_col_types = FALSE) |>
  mutate(
    Date = as.Date(parse_date_time(Date, orders = c("dmy", "mdy", "ymd"))),
    Price = as.numeric(gsub(",", "", as.character(Price)))
  ) |>
  select(Date, TTF = Price) |>
  filter(!is.na(Date), !is.na(TTF)) |>
  arrange(Date) |>
  distinct(Date, .keep_all = TRUE)

cat("TTF      :", nrow(ttf), "obs. (",
    as.character(min(ttf$Date)), "-", as.character(max(ttf$Date)), ")\n")
```

```
## TTF      : 1790 obs. ( 2017-10-23 - 2024-11-29 )
```

3.6 Scalenie i raport jakości

Łączymy po kalendarzu TTF (sesje giełdowe). Krótkie luki (weekendy, święta) uzupełniamy LOCF z limitem 5 dni.

```
df <- ttf |>
  left_join(brent |> select(Date, Brent), by = "Date") |>
  left_join(eurusd |> select(Date, EURUSD), by = "Date")

if (!is.null(storage)) {
  df <- df |> left_join(storage |> select(Date, Storage_pct), by = "Date")
}
if (!is.null(weather)) {
  df <- df |> left_join(weather |> select(Date, T_mean_EU, HDD), by = "Date")
}

locf_cols <- intersect(c("Brent", "EURUSD", "Storage_pct", "T_mean_EU", "HDD"),
  names(df))
df <- df |>
  mutate(across(all_of(locf_cols), ~ zoo::na.locf(.x, maxgap = 5, na.rm = FALSE)))

# Zostawiamy tylko wiersze z kompletem zmiennych dziennych
core_cols <- intersect(c("TTF", "Brent", "EURUSD"), names(df))
df <- df |> filter(if_all(all_of(core_cols), ~ !is.na(.x)))

quality <- df |>
  summarise(across(everything(), ~ sum(is.na(.x)))) |>
  pivot_longer(everything(), names_to = "Zmienna", values_to = "NA_count") |>
  mutate(NA_pct = round(100 * NA_count / nrow(df), 2))

kable(quality, caption = "Braki danych w finalnym panelu", booktabs = TRUE) |>
  kable_styling(latex_options = c("striped", "HOLD_position"), full_width = FALSE)
```

Tabela 1: Braki danych w finalnym panelu

Zmienna	NA_count	NA_pct
Date	0	0
TTF	0	0
Brent	0	0
EURUSD	0	0
Storage_pct	0	0

```
write_csv(df, file.path(PROC_DIR, "ttf_merged.csv"))

cat("\nFinalny panel:", nrow(df), "obs.",
    as.character(min(df$Date)), "-", as.character(max(df$Date)), "\n")

##
## Finalny panel: 1790 obs., 2017-10-23 - 2024-11-29
```

3.7 Statystyki opisowe

```
num_cols <- df |> select(where(is.numeric)) |> names()

stats <- df |>
  select(all_of(num_cols)) |>
  summarise(across(everything(), list(
    Srednia = ~ mean(.x, na.rm = TRUE),
    Mediana = ~ median(.x, na.rm = TRUE),
    Odch_std = ~ sd(.x, na.rm = TRUE),
    Min = ~ min(.x, na.rm = TRUE),
    Max = ~ max(.x, na.rm = TRUE),
    Skosnosc = ~ moments::skewness(.x, na.rm = TRUE),
    Kurtoza = ~ moments::kurtosis(.x, na.rm = TRUE)
  ))) |>
  pivot_longer(everything(),
    names_to = c("Zmienna", ".value"),
    names_sep = "_(?=[A-Z])")

kable(stats, digits = 2, caption = "Statystyki opisowe", booktabs = TRUE) |>
  kable_styling(latex_options = c("striped", "HOLD_position"))
```

Tabela 2: Statystyki opisowe

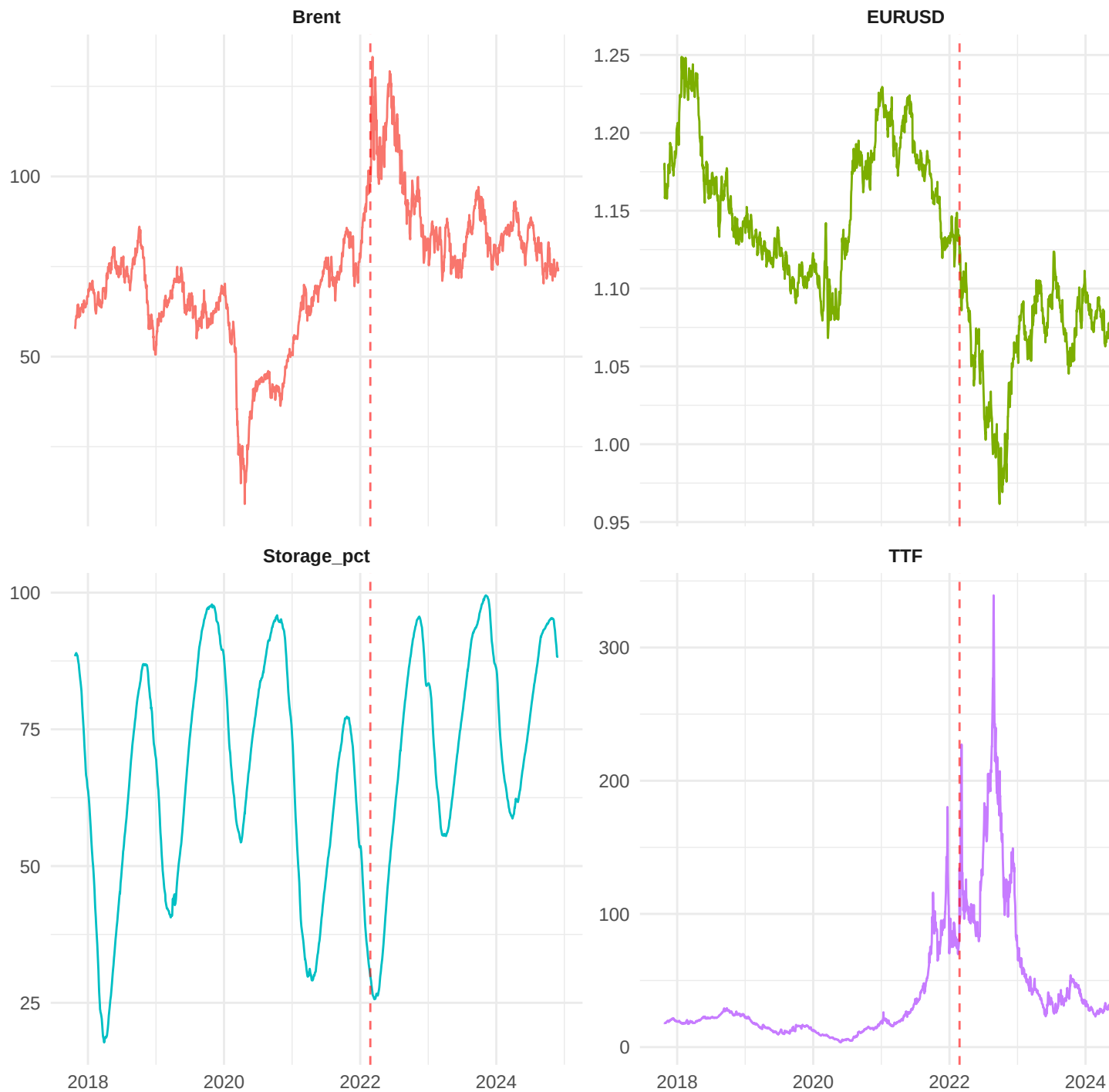
Zmienna	Srednia	Mediana	Odch_std	Min	Max	Skosnosc	Kurtoza
TTF	42.62	26.01	45.43	3.51	339.20	2.43	9.73
Brent	72.79	73.72	18.89	9.12	133.18	-0.12	4.00
EURUSD	1.12	1.12	0.06	0.96	1.25	-0.03	2.67
Storage_pct	68.25	70.88	21.27	17.75	99.50	-0.48	2.22

4 Analiza eksploracyjna (EDA)

4.1 Wykresy szeregów

```
df |>
  pivot_longer(-Date, names_to = "Zmienna", values_to = "Wartosc") |>
  ggplot(aes(Date, Wartosc, color = Zmienna)) +
  geom_line(linewidth = 0.5, show.legend = FALSE) +
  facet_wrap(~ Zmienna, scales = "free_y", ncol = 2) +
  geom_vline(xintercept = as.Date("2022-02-24"),
             linetype = "dashed", color = "red", alpha = 0.6) +
  labs(title = "Szeregi czasowe: TTF i regresory fundamentalne",
       subtitle = "Czerwona linia: inwazja Rosji na Ukraine (24.02.2022)",
       x = NULL, y = NULL) +
  theme(strip.text = element_text(face = "bold"))
```

Szeregi czasowe: TTF i regresory fundamentalne
Czerwona linia: inwazja Rosji na Ukrainę (24.02.2022)



Rysunek 2: Ewolucja wszystkich analizowanych szeregów

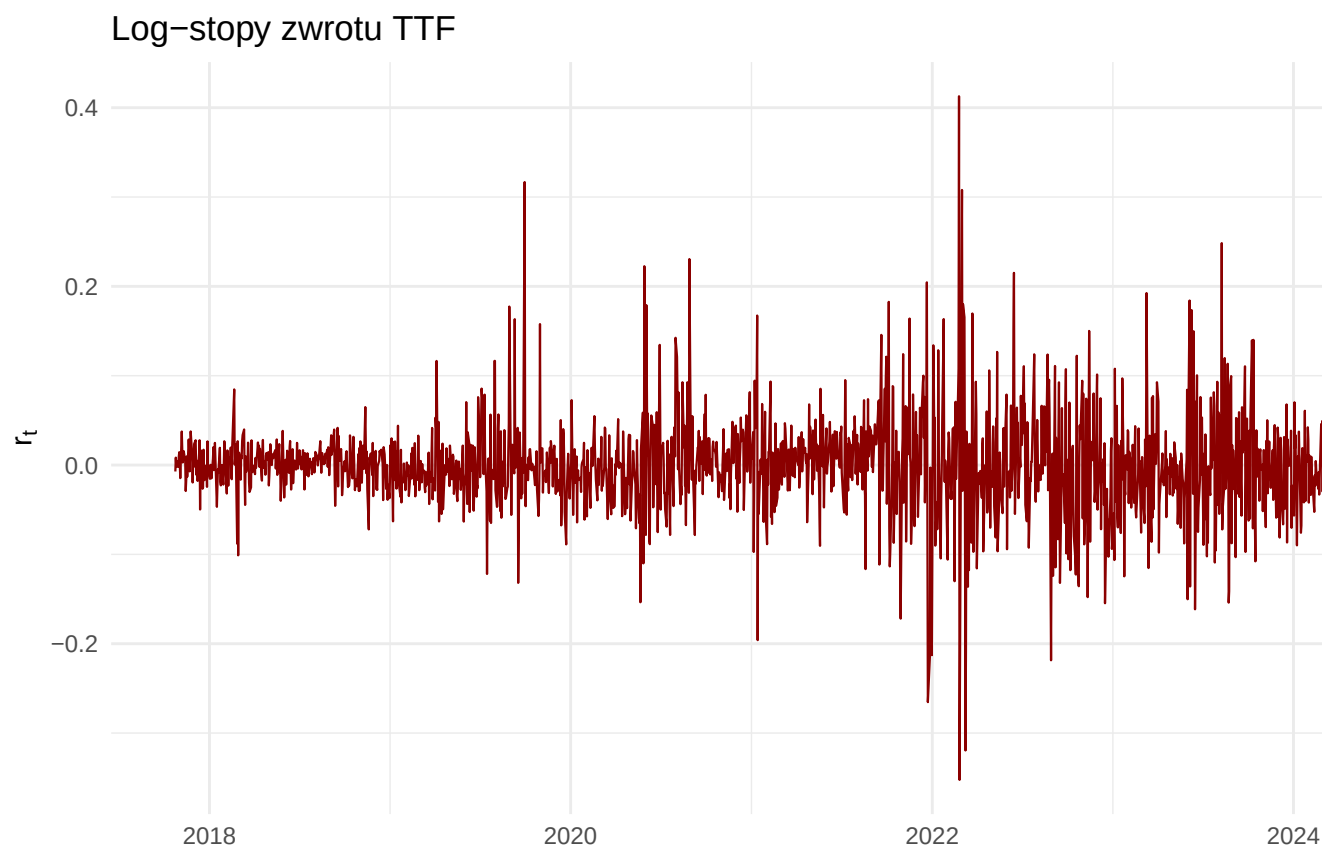
4.2 Log-stopu zwrotu i pierwsze różnice

```
has_storage <- "Storage_pct" %in% names(df)
has_hdd     <- "HDD"         %in% names(df)

df_diff <- df |>
  mutate(
    r_TTF      = c(NA, diff(log(TTF))),
    r_Brent    = c(NA, diff(log(Brent))),
    r_EURUSD   = c(NA, diff(log(EURUSD))),
    d_Storage  = if (has_storage) c(NA, diff(Storage_pct)) else NA_real_,
    d_HDD      = if (has_hdd)     c(NA, diff(HDD))         else NA_real_
  ) |>
  filter(if_all(c(r_TTF, r_Brent, r_EURUSD), ~ !is.na(.x)))

returns <- df_diff$r_TTF
```

```
ggplot(df_diff, aes(Date, r_TTF)) +
  geom_line(color = "darkred", linewidth = 0.4) +
  labs(title = "Log-stopu zwrotu TTF", x = NULL, y = expression(r[t]))
```



Rysunek 3: Logarytmiczne stopy zwrotu TTF — widoczne klastry zmienności

```
ret_stats <- tibble(
  Srednia      = mean(returns),
  Odch_std     = sd(returns),
  Min          = min(returns),
  Max          = max(returns),
  Skosnosc     = moments::skewness(returns),
  Kurtoza_nadwyzkowa = moments::kurtosis(returns) - 3,
  JB_pvalue    = jarque.bera.test(returns)$p.value
)

kable(ret_stats, digits = 4, caption = "Statystyki log-stop zwrotu TTF",
      booktabs = TRUE)
```

Tabela 3: Statystyki log-stop zwrotu TTF

Srednia	Odch_std	Min	Max	Skosnosc	Kurtoza_nadwyzkowa	JB_pvalue
5e-04	0.0526	-0.3524	0.4128	0.378	7.5557	0

4.3 Korelacje

```
cor_levels <- df |> select(all_of(num_cols)) |> cor(use = "pairwise.complete.obs")

ret_cols <- intersect(c("r_TTF", "r_Brent", "r_EURUSD", "d_Storage", "d_HDD"),
                     names(df_diff))
cor_diffs <- df_diff |> select(all_of(ret_cols)) |>
  cor(use = "pairwise.complete.obs")

kable(round(cor_levels, 3),
      caption = "Korelacje w poziomach (uwaga: szeregi I(1), mozliwe pozorne)",
      booktabs = TRUE)
```

Tabela 4: Korelacje w poziomach (uwaga: szeregi I(1), mozliwe pozorne)

	TTF	Brent	EURUSD	Storage_pct
TTF	1.000	0.651	-0.549	-0.002
Brent	0.651	1.000	-0.460	-0.157
EURUSD	-0.549	-0.460	1.000	-0.377
Storage_pct	-0.002	-0.157	-0.377	1.000

```
kable(round(cor_diffs, 3),
      caption = "Korelacje stop zwrotu / pierwszych roznic (krotkookresowe)",
      booktabs = TRUE)
```

Tabela 5: Korelacje stop zwrotu / pierwszych roznic (krotkookresowe)

	r_TTF	r_Brent	r_EURUSD	d_Storage	d_HDD
r_TTF	1.000	0.115	-0.107	0.052	NA
r_Brent	0.115	1.000	0.011	-0.006	NA
r_EURUSD	-0.107	0.011	1.000	-0.007	NA
d_Storage	0.052	-0.006	-0.007	1.000	NA
d_HDD	NA	NA	NA	NA	NA

5 Testy stacjonarności

H_0 (ADF): szereg jest niestacjonarny. H_0 (KPSS): szereg jest stacjonarny.

```
do_tests <- function(x, name) {
  x <- na.omit(x)
  adf <- suppressWarnings(adf.test(x))
  kpss <- suppressWarnings(kpss.test(x))
  tibble(
    Zmienna      = name,
    ADF_stat     = round(adf$statistic, 3),
    ADF_pvalue   = round(adf$p.value, 3),
    KPSS_stat    = round(kpss$statistic, 3),
    KPSS_pvalue  = round(kpss$p.value, 3)
  )
}

vars_levels <- list(
  "log(TTF)"    = log(df$TTF),
  "log(Brent)"  = log(df$Brent),
  "log(EURUSD)" = log(df$EURUSD)
)
if (has_hdd)    vars_levels[["HDD"]] <- df$HDD
if (has_storage) vars_levels[["Storage_pct"]] <- df$Storage_pct

vars_diffs <- list(
  "d log(TTF)"   = df_diff$r_TTF,
  "d log(Brent)" = df_diff$r_Brent,
  "d log(EURUSD)" = df_diff$r_EURUSD
)
if (has_hdd)    vars_diffs[["d HDD"]] <- df_diff$d_HDD
if (has_storage) vars_diffs[["d Storage_pct"]] <- df_diff$d_Storage

levels_tests <- imap_dfr(vars_levels, ~ do_tests(.x, .y))
diffs_tests  <- imap_dfr(vars_diffs, ~ do_tests(.x, .y))

kable(levels_tests, caption = "Stacjonarnosc - poziomy", booktabs = TRUE) |>
  kable_styling(latex_options = c("striped", "HOLD_position"))
```

Tabela 6: Stacjonarnosc - poziomy

Zmienna	ADF_stat	ADF_pvalue	KPSS_stat	KPSS_pvalue
log(TTF)	-1.398	0.833	7.953	0.01
log(Brent)	-2.276	0.462	5.526	0.01
log(EURUSD)	-2.025	0.568	7.925	0.01
Storage_pct	-6.128	0.010	1.681	0.01

```
kable(diffs_tests, caption = "Stacjonarnosc - pierwsze roznice", booktabs = TRUE) |>
  kable_styling(latex_options = c("striped", "HOLD_position"))
```

Tabela 7: Stacjonarnosc - pierwsze roznice

Zmienna	ADF_stat	ADF_pvalue	KPSS_stat	KPSS_pvalue
d log(TTF)	-12.565	0.010	0.116	0.1
d log(Brent)	-11.032	0.010	0.049	0.1
d log(EURUSD)	-12.414	0.010	0.052	0.1
d Storage_pct	-3.635	0.029	0.233	0.1

Wniosek: szeregi cenowe są $I(1)$, pierwsze różnice stacjonarne — **H1 potwierdzona**.

6 Analiza kointegracji

6.1 Engle-Granger (TTF vs Brent)

Regresja statyczna $\log TTF_t = \alpha + \beta \log Brent_t + u_t$, następnie test ADF na resztach.

```
eg_reg <- lm(log(TTF) ~ log(Brent), data = df)
summary(eg_reg)
```

```
##
## Call:
## lm(formula = log(TTF) ~ log(Brent), data = df)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -1.0231 -0.3948 -0.1126  0.2857  2.8073
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept) -5.65580    0.17778  -31.81  <2e-16 ***
## log(Brent)   2.12238    0.04176   50.83  <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.5481 on 1788 degrees of freedom
## Multiple R-squared:  0.591, Adjusted R-squared:  0.5908
## F-statistic: 2584 on 1 and 1788 DF, p-value: < 2.2e-16
```



```
adf_eg <- adf.test(residuals(eg_reg))
print(adf_eg)
```

```
##
## Augmented Dickey-Fuller Test
##
## data: residuals(eg_reg)
## Dickey-Fuller = -3.3012, Lag order = 12, p-value = 0.07046
## alternative hypothesis: stationary
```

6.2 Johansen (TTF, Brent, EUR/USD)

```
johansen_data <- df |>
  transmute(log_TTF = log(TTF), log_Brent = log(Brent), log_EURUSD = log(EURUSD)) |>
  as.matrix()
```

```
joh <- ca.jo(johansen_data, type = "trace", ecdet = "const", K = 2)
summary(joh)
```

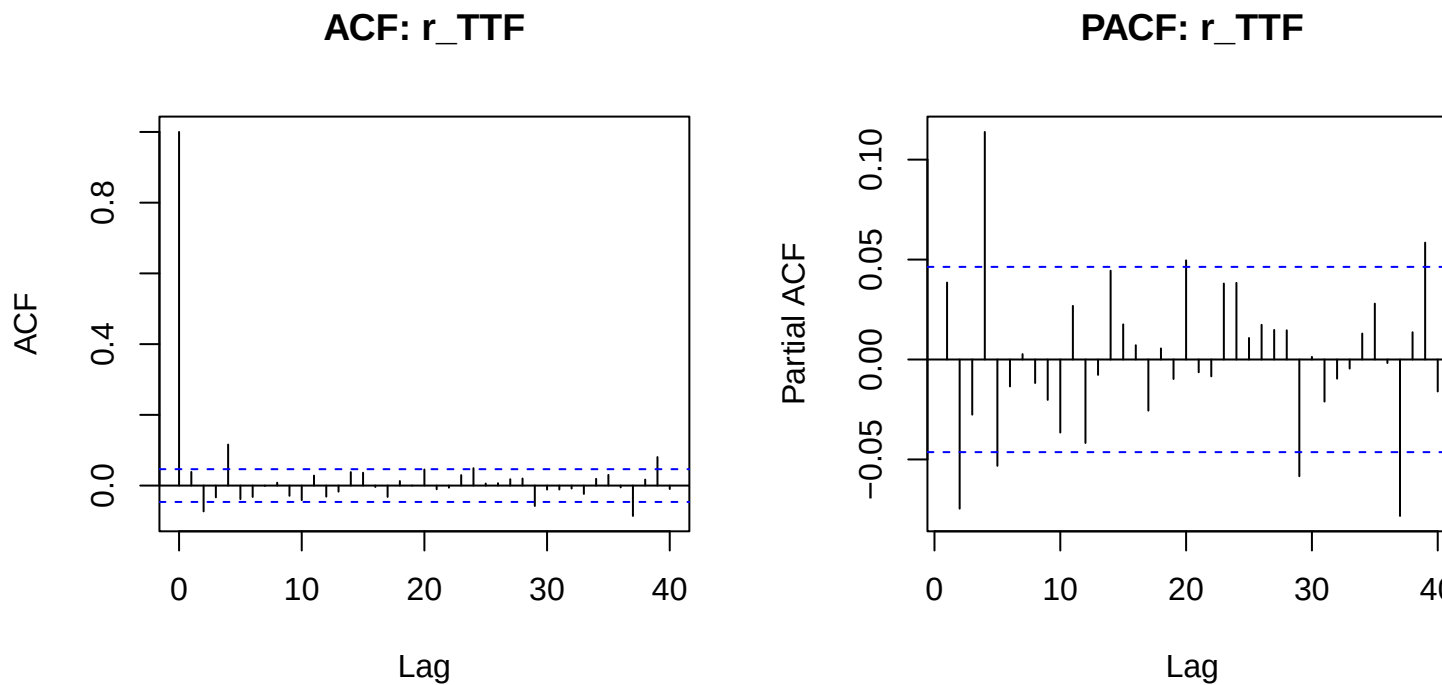
```
##
## #####
## # Johansen-Procedure #
## #####
##
## Test type: trace statistic , without linear trend and constant in cointegration
##
## Eigenvalues (lambda):
## [1] 0.008857632 0.006632453 0.002304946 0.000000000
##
## Values of teststatistic and critical values of test:
##
##          test 10pct  5pct  1pct
## r <= 2 |   4.13   7.52   9.24 12.97
## r <= 1 |  16.02  17.85  19.96 24.60
## r = 0  |  31.93  32.00  34.91 41.07
##
## Eigenvectors, normalised to first column:
## (These are the cointegration relations)
##
##          log_TTF.12 log_Brent.12 log_EURUSD.12  constant
## log_TTF.12      1.000000      1.000000      1.000000      1.000000
## log_Brent.12    -4.590779     -1.636728      0.8746197   0.5691903
## log_EURUSD.12 -10.933589     12.689853     -5.0355828  40.2169208
## constant       17.274988      2.244967     -6.5346977 -19.5085140
##
## Weights W:
## (This is the loading matrix)
##
##          log_TTF.12 log_Brent.12 log_EURUSD.12  constant
## log_TTF.d    -0.0032931543  0.0006429902 -1.456689e-03  6.938349e-20
## log_Brent.d   0.0015535325  0.0026136788 -7.746046e-04 -7.626163e-19
```

```
## log_EURUSD.d  0.0001863115 -0.0004147812 -7.409509e-05 -8.899601e-20
```

Najmniejsze r , dla którego *nie odrzucamy* $H_0 : r \leq r$, jest liczbą wektorów kointegrujących. Co najmniej 1 wektor potwierdza **H2** i uzasadnia obecność Brent jako “kotwicy cenowej” w ARIMAX.

7 Identyfikacja ARIMA

```
par(mfrow = c(1, 2))
acf(returns, main = "ACF: r_TTF", lag.max = 40)
pacf(returns, main = "PACF: r_TTF", lag.max = 40)
```



Rysunek 4: ACF i PACF log-stop zwrotu TTF

```
par(mfrow = c(1, 1))
```

8 Estymacja ARIMA i ARIMAX

8.1 Model bazowy: ARIMA jednowymiarowy

```
fit_arima <- auto.arima(
  log(df$TTF),
```

```

d = 1, seasonal = FALSE,
stepwise = FALSE, approximation = FALSE, ic = "aicc"
)
summary(fit_arima)

## Series: log(df$TTF)
## ARIMA(5,1,0)
##
## Coefficients:
##          ar1      ar2      ar3      ar4      ar5
##      0.0486 -0.0666 -0.0353  0.1160 -0.0529
## s.e.  0.0236  0.0235  0.0235  0.0234  0.0236
##
## sigma^2 = 0.00271:  log likelihood = 2751.22
## AIC=-5490.44  AICc=-5490.39  BIC=-5457.51
##
## Training set error measures:
##              ME      RMSE      MAE      MPE      MAPE      MASE
## Training set 0.0005330637 0.05196973 0.03521682 0.002997964 1.074111 0.997455
##              ACF1
## Training set -0.0007983086

```

8.2 Model rozszerzony: ARIMAX

Regresory: $\log \text{Brent}$, $\log \text{EURUSD}$, Storage_pct , HDD .

```

xreg_list <- list(
  log_Brent = log(df$Brent),
  log_EURUSD = log(df$EURUSD)
)
if (has_storage) xreg_list$Storage_pct <- df$Storage_pct
if (has_hdd) xreg_list$HDD <- df$HDD

xreg_full <- as.matrix(as.data.frame(xreg_list))
cat("Regresory w ARIMAX:", paste(colnames(xreg_full), collapse = ", "), "\n")

```

```
## Regresory w ARIMAX: log_Brent, log_EURUSD, Storage_pct
```

```

fit_arimax <- auto.arima(
  log(df$TTF),
  xreg = xreg_full,
  d = 1, seasonal = FALSE,
  stepwise = FALSE, approximation = FALSE, ic = "aicc"
)
summary(fit_arimax)

```

```

## Series: log(df$TTF)
## Regression with ARIMA(5,1,0) errors
##
## Coefficients:
##          ar1      ar2      ar3      ar4      ar5 log_Brent log_EURUSD

```

```
##      0.0447 -0.0697 -0.0402 0.1095 -0.0489      0.1716      -1.2563
## s.e. 0.0236  0.0235  0.0235 0.0235  0.0236      0.0360      0.2740
##      Storage_pct
##              0.0048
## s.e.      0.0020
##
## sigma^2 = 0.002643: log likelihood = 2775.25
## AIC=-5532.51  AICc=-5532.41  BIC=-5483.1
##
## Training set error measures:
##              ME      RMSE      MAE      MPE      MAPE      MASE
## Training set 0.0004391665 0.05127633 0.03497829 0.0006451362 1.072543 0.9906991
##              ACF1
## Training set -0.0008396013
```

8.3 Porównanie ARIMA vs ARIMAX

```
cmp <- tibble(
  Model      = c("ARIMA(p,1,q)", "ARIMAX(p,1,q)"),
  AICc       = c(fit_arma$aicc, fit_arimax$aicc),
  BIC        = c(fit_arma$bic, fit_arimax$bic),
  log_Lik    = c(fit_arma$loglik, fit_arimax$loglik),
  Liczba_par = c(length(fit_arma$coef), length(fit_arimax$coef))
)

kable(cmp, digits = 2, caption = "Porownanie modeli", booktabs = TRUE)
```

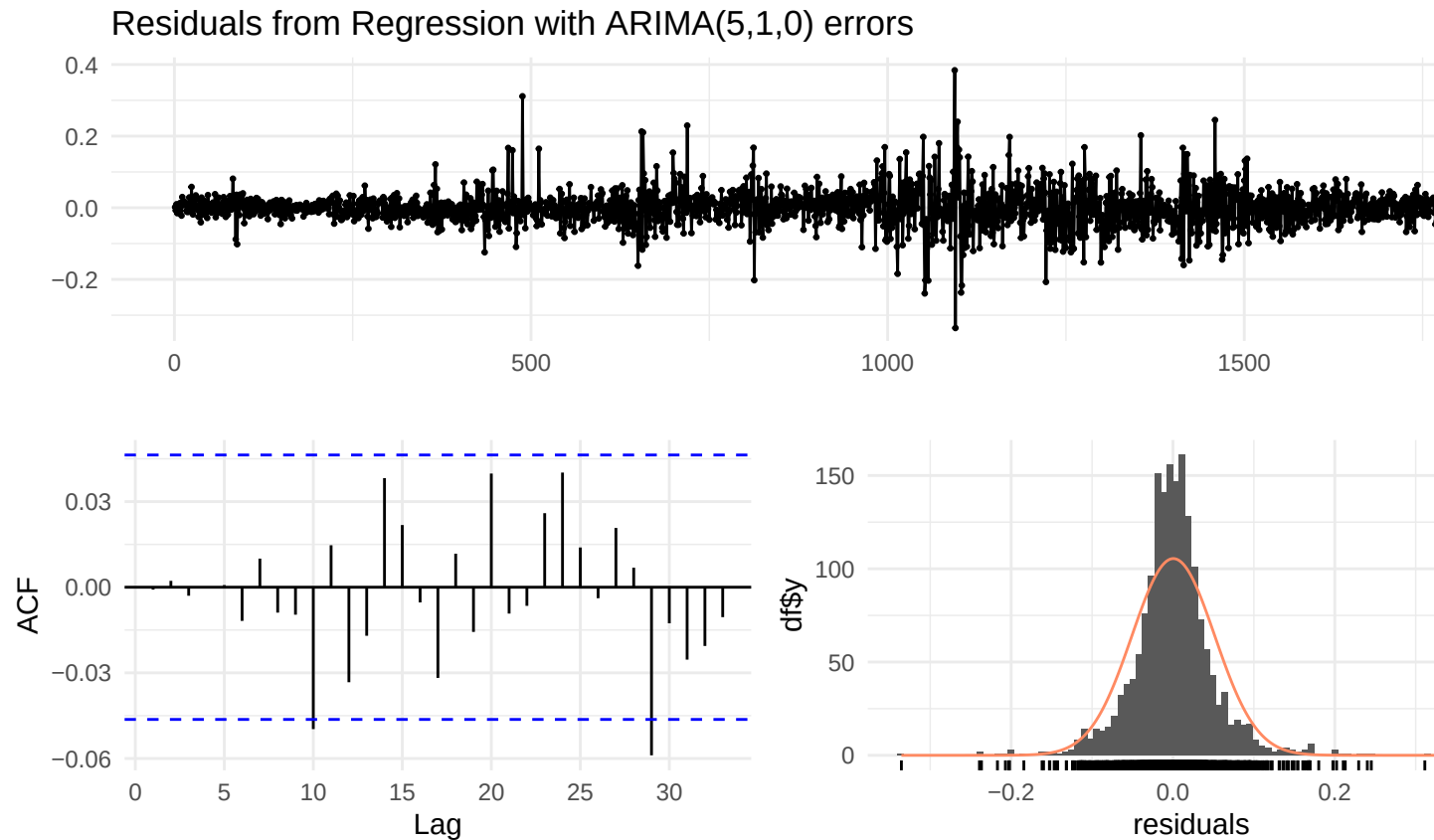
Tabela 8: Porownanie modeli

Model	AICc	BIC	log_Lik	Liczba_par
ARIMA(p,1,q)	-5490.39	-5457.51	2751.22	5
ARIMAX(p,1,q)	-5532.41	-5483.10	2775.25	8

Współczynniki na regresorach w “regresji z błędami ARIMA” interpretujemy jako długookresowe wrażliwości log-ceny TTF. Ujemny współczynnik przy `Storage_pct` potwierdza: wzrost zapasów o 1 p.p. obniża log-cenę TTF.

8.4 Diagnostyka reszt ARIMAX

```
arimax_resid <- residuals(fit_arimax)
checkresiduals(fit_arimax)
```



Rysunek 5: Diagnostyka reszt ARIMAX

```
##
##  Ljung-Box test
##
## data:  Residuals from Regression with ARIMA(5,1,0) errors
## Q* = 5.2203, df = 5, p-value = 0.3896
##
## Model df: 5.   Total lags used: 10
```

```
lb <- Box.test(arimax_resid, lag = 20, type = "Ljung-Box",
               fitdf = sum(fit_arimax$arima[1:2]))
print(lb)
```

```
##
##  Box-Ljung test
##
## data:  arimax_resid
## X-squared = 17.073, df = 15, p-value = 0.3145
```

```
jb <- jarque.bera.test(arimax_resid)
print(jb)
```

```
##
```

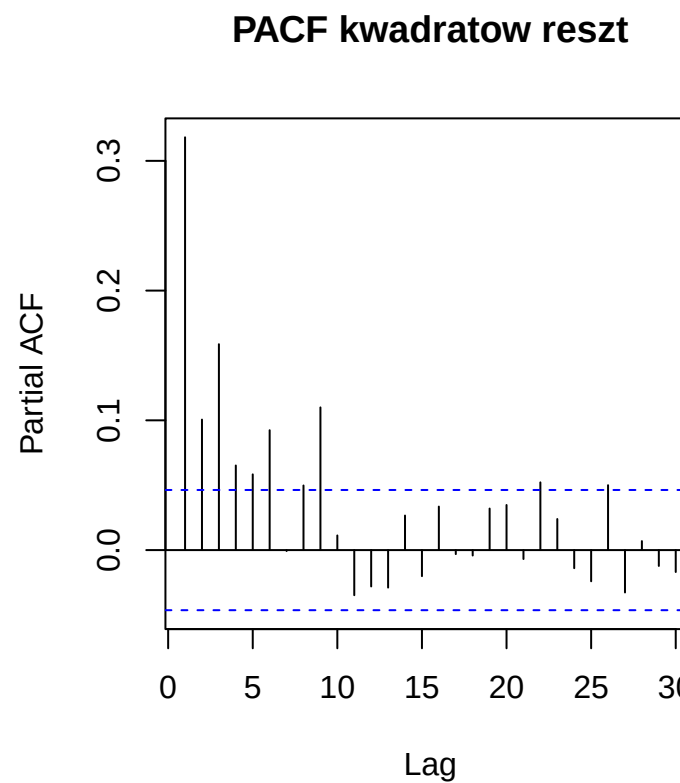
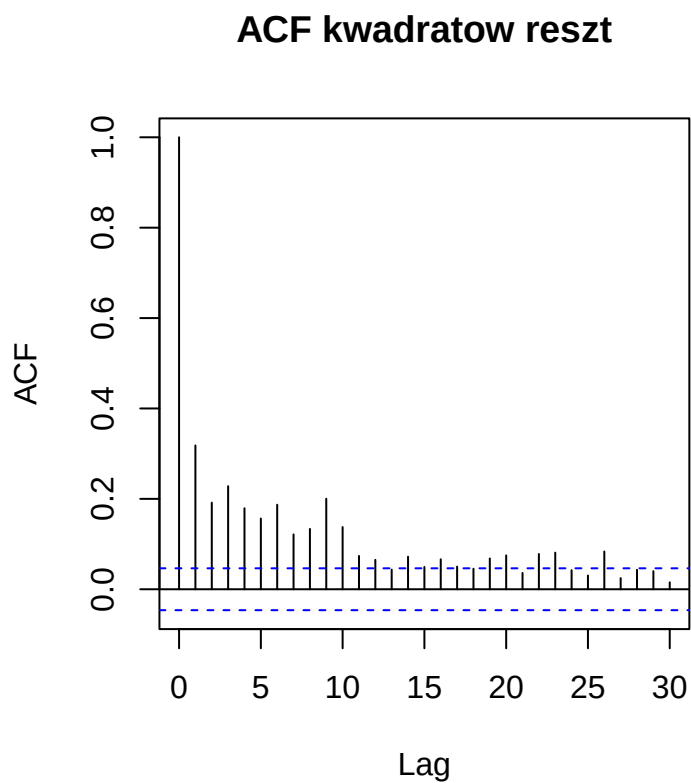
```
## Jarque Bera Test
##
## data: arimax_resid
## X-squared = 2894.3, df = 2, p-value < 2.2e-16
```

9 Test efektu ARCH

```
arch_test <- ArchTest(arimax_resid, lags = 12)
print(arch_test)
```

```
##
## ARCH LM-test; Null hypothesis: no ARCH effects
##
## data: arimax_resid
## Chi-squared = 285.53, df = 12, p-value < 2.2e-16
```

```
par(mfrow = c(1, 2))
acf(arimax_resid^2, main = "ACF kwadratow reszt", lag.max = 30)
pacf(arimax_resid^2, main = "PACF kwadratow reszt", lag.max = 30)
```



Rysunek 6: ACF i PACF kwadratow reszt ARIMAX

```
par(mfrow = c(1, 1))
```

Jeśli $p < 0.05$ — **H3** potwierdzona.

10 Estymacja modelu GARCH-X

W równaniu średniej: ARMA z regresorami zwrotów Brent/EUR-USD oraz zmianami zapasów i HDD. W równaniu wariancji: poziom Storage_pct (intuicja: niskie zapasy → wyższa zmienność).

```
returns_xts <- xts(df_diff$r_TTF, order.by = df_diff$Date)

ext_mean_cols <- c("r_Brent", "r_EURUSD")
if (has_storage) ext_mean_cols <- c(ext_mean_cols, "d_Storage")
if (has_hdd)      ext_mean_cols <- c(ext_mean_cols, "d_HDD")
ext_mean <- as.matrix(df_diff[, ext_mean_cols])

ext_var <- if (has_storage) {
  as.matrix(df_diff[, "Storage_pct", drop = FALSE])
} else {
  NULL
}
cat("Regresory w sredniej GARCH:", paste(ext_mean_cols, collapse = ", "), "\n")
```

```
## Regresory w sredniej GARCH: r_Brent, r_EURUSD, d_Storage
```

```
cat("Regresor w wariancji GARCH:",
    if (has_storage) "Storage_pct" else "(brak)", "\n")
```

```
## Regresor w wariancji GARCH: Storage_pct
```

```
arma_order <- c(fit_arimax$arma[1], fit_arimax$arma[2])
cat("Rzedy ARMA w GARCH:", arma_order, "\n")
```

```
## Rzedy ARMA w GARCH: 5 0
```

10.1 sGARCH(1,1) bazowy (bez X)

```
spec_base <- ugarchspec(
  variance.model = list(model = "sGARCH", garchOrder = c(1, 1)),
  mean.model     = list(armaOrder = arma_order, include.mean = TRUE),
  distribution.model = "std"
)
fit_garch_base <- ugarchfit(spec_base, data = returns_xts, solver = "hybrid")
fit_garch_base
```

```

##
## *-----*
## *          GARCH Model Fit          *
## *-----*
##
## Conditional Variance Dynamics
## -----
## GARCH Model   : sGARCH(1,1)
## Mean Model    : ARFIMA(5,0,0)
## Distribution   : std
##
## Optimal Parameters
## -----
##      Estimate  Std. Error   t value Pr(>|t|)
## mu      0.000300   0.000797   0.376490 0.706552
## ar1     0.029890   0.024735   1.208396 0.226895
## ar2    -0.039513   0.024424  -1.617752 0.105716
## ar3    -0.009356   0.024088  -0.388428 0.697699
## ar4     0.058179   0.024034   2.420668 0.015492
## ar5    -0.001698   0.024107  -0.070421 0.943858
## omega   0.000031   0.000011   2.678345 0.007399
## alpha1  0.156018   0.023109   6.751298 0.000000
## beta1   0.842982   0.021014  40.115910 0.000000
## shape   6.504211   0.898245   7.241021 0.000000
##
## Robust Standard Errors:
##      Estimate  Std. Error   t value Pr(>|t|)
## mu      0.000300   0.001054   0.284642 0.775918
## ar1     0.029890   0.022274   1.341880 0.179635
## ar2    -0.039513   0.022522  -1.754440 0.079355
## ar3    -0.009356   0.021417  -0.436869 0.662207
## ar4     0.058179   0.023210   2.506643 0.012188
## ar5    -0.001698   0.024875  -0.068246 0.945590
## omega   0.000031   0.000017   1.831288 0.067058
## alpha1  0.156018   0.025936   6.015550 0.000000
## beta1   0.842982   0.026349  31.992399 0.000000
## shape   6.504211   1.060777   6.131554 0.000000
##
## LogLikelihood : 3181.269
##
## Information Criteria
## -----
##
## Akaike          -3.5453
## Bayes           -3.5146
## Shibata         -3.5454
## Hannan-Quinn   -3.5340
##
## Weighted Ljung-Box Test on Standardized Residuals
## -----
##                                statistic p-value
## Lag[1]                                1.056  0.3041
## Lag[2*(p+q)+(p+q)-1][14]            3.318  1.0000
## Lag[4*(p+q)+(p+q)-1][24]           11.672  0.5983

```



```

## d.o.f=5
## H0 : No serial correlation
##
## Weighted Ljung-Box Test on Standardized Squared Residuals
## -----
##
##              statistic p-value
## Lag[1]              0.0476 0.8273
## Lag[2*(p+q)+(p+q)-1][5] 0.3112 0.9825
## Lag[4*(p+q)+(p+q)-1][9] 1.9608 0.9096
## d.o.f=2
##
## Weighted ARCH LM Tests
## -----
##
##      Statistic Shape Scale P-Value
## ARCH Lag[3]    0.1859 0.500 2.000 0.6663
## ARCH Lag[5]    0.2864 1.440 1.667 0.9439
## ARCH Lag[7]    1.4813 2.315 1.543 0.8251
##
## Nyblom stability test
## -----
## Joint Statistic: 4.0332
## Individual Statistics:
## mu      0.27316
## ar1     0.35324
## ar2     0.07923
## ar3     0.20500
## ar4     0.22251
## ar5     0.10881
## omega   1.47744
## alpha1  0.47744
## beta1   0.63865
## shape   1.45202
##
## Asymptotic Critical Values (10% 5% 1%)
## Joint Statistic:      2.29 2.54 3.05
## Individual Statistic: 0.35 0.47 0.75
##
## Sign Bias Test
## -----
##
##              t-value   prob sig
## Sign Bias      0.005208 0.99585
## Negative Sign Bias 1.573096 0.11587
## Positive Sign Bias 1.439648 0.15014
## Joint Effect    8.400606 0.03842 **
##
##
## Adjusted Pearson Goodness-of-Fit Test:
## -----
##      group statistic p-value(g-1)
## 1      20      11.88      0.8906
## 2      30      28.56      0.4883
## 3      40      37.47      0.5396
## 4      50      57.81      0.1818
##

```

```
##
## Elapsed time : 0.4399981
```

10.2 sGARCH(1,1)-X — pełny

```
spec_x <- ugarchspec(
  variance.model = list(
    model = "sGARCH",
    garchOrder = c(1, 1),
    external.regressors = ext_var
  ),
  mean.model = list(
    armaOrder = arma_order,
    include.mean = TRUE,
    external.regressors = ext_mean
  ),
  distribution.model = "std"
)
fit_garch_x <- ugarchfit(spec_x, data = returns_xts, solver = "hybrid")
fit_garch_x
```

```
##
## *-----*
## *          GARCH Model Fit          *
## *-----*
##
## Conditional Variance Dynamics
## -----
## GARCH Model   : sGARCH(1,1)
## Mean Model    : ARFIMA(5,0,0)
## Distribution   : std
##
## Optimal Parameters
## -----
##      Estimate Std. Error  t value Pr(>|t|)
## mu      0.000000   0.000718  0.000386 0.999692
## ar1     0.024304   0.024488  0.992500 0.320954
## ar2    -0.048968   0.024508 -1.998010 0.045716
## ar3    -0.010342   0.023226 -0.445302 0.656101
## ar4     0.048513   0.021162  2.292463 0.021879
## ar5    -0.003128   0.022795 -0.137224 0.890854
## mxreg1  0.172544   0.031504  5.476912 0.000000
## mxreg2 -0.423172   0.114870 -3.683921 0.000230
## mxreg3  0.005497   0.001167  4.711424 0.000002
## omega   0.000000   0.000009  0.000006 0.999995
## alpha1  0.171936   0.024590  6.992180 0.000000
## beta1   0.838444   0.017932 46.757026 0.000000
## vxreg1  0.000000   0.000000  7.990976 0.000000
## shape   6.630697   0.948072  6.993872 0.000000
##
## Robust Standard Errors:
##      Estimate Std. Error  t value Pr(>|t|)
```

```

## mu      0.000000    0.000874  0.000317  0.999747
## ar1     0.024304    0.022436  1.083243  0.278701
## ar2    -0.048968    0.022591 -2.167609  0.030188
## ar3    -0.010342    0.022109 -0.467783  0.639940
## ar4     0.048513    0.022328  2.172759  0.029798
## ar5    -0.003128    0.023695 -0.132011  0.894976
## mxreg1  0.172544    0.070740  2.439122  0.014723
## mxreg2 -0.423172    0.129592 -3.265409  0.001093
## mxreg3  0.005497    0.001354  4.059850  0.000049
## omega   0.000000    0.000007  0.000007  0.999994
## alpha1  0.171936    0.028690  5.992932  0.000000
## beta1   0.838444    0.021039 39.852257  0.000000
## vxreg1  0.000000    0.000000  6.877649  0.000000
## shape   6.630697    1.192674  5.559521  0.000000
##
## LogLikelihood : 3216.184
##
## Information Criteria
## -----
##
## Akaike      -3.5799
## Bayes      -3.5369
## Shibata    -3.5800
## Hannan-Quinn -3.5640
##
## Weighted Ljung-Box Test on Standardized Residuals
## -----
##
##                statistic p-value
## Lag[1]                1.247  0.2642
## Lag[2*(p+q)+(p+q)-1][14]  4.324  1.0000
## Lag[4*(p+q)+(p+q)-1][24] 11.228  0.6650
## d.o.f=5
## H0 : No serial correlation
##
## Weighted Ljung-Box Test on Standardized Squared Residuals
## -----
##
##                statistic p-value
## Lag[1]                0.002303  0.9617
## Lag[2*(p+q)+(p+q)-1][5]  0.723856  0.9183
## Lag[4*(p+q)+(p+q)-1][9]  2.537776  0.8321
## d.o.f=2
##
## Weighted ARCH LM Tests
## -----
##
##                Statistic Shape Scale P-Value
## ARCH Lag[3]      0.4321 0.500 2.000  0.5109
## ARCH Lag[5]      0.6392 1.440 1.667  0.8419
## ARCH Lag[7]      2.0739 2.315 1.543  0.7017
##
## Nyblom stability test
## -----
## Joint Statistic: 10.3502
## Individual Statistics:
## mu      0.25630

```

```

## ar1      0.30733
## ar2      0.09413
## ar3      0.17240
## ar4      0.23453
## ar5      0.12246
## mxreg1 0.36946
## mxreg2 0.15151
## mxreg3 0.08190
## omega 2.39870
## alpha1 0.32784
## beta1 0.39528
## vxreg1 4.15957
## shape 0.99324
##
## Asymptotic Critical Values (10% 5% 1%)
## Joint Statistic:          3.08 3.34 3.9
## Individual Statistic:    0.35 0.47 0.75
##
## Sign Bias Test
## -----
##              t-value    prob sig
## Sign Bias          0.2937 0.76906
## Negative Sign Bias 1.8875 0.05926 *
## Positive Sign Bias 1.0729 0.28347
## Joint Effect       7.0983 0.06883 *
##
##
## Adjusted Pearson Goodness-of-Fit Test:
## -----
##   group statistic p-value(g-1)
## 1    20      24.11    0.19184
## 2    30      25.07    0.67465
## 3    40      52.05    0.07886
## 4    50      49.37    0.45819
##
##
## Elapsed time : 0.8524652

```

10.3 Diagnostyka GARCH-X

```

std_resid_x <- as.numeric(residuals(fit_garch_x, standardize = TRUE))
ArchTest(std_resid_x, lags = 12)

```

```

##
## ARCH LM-test; Null hypothesis: no ARCH effects
##
## data:  std_resid_x
## Chi-squared = 8.1153, df = 12, p-value = 0.7761

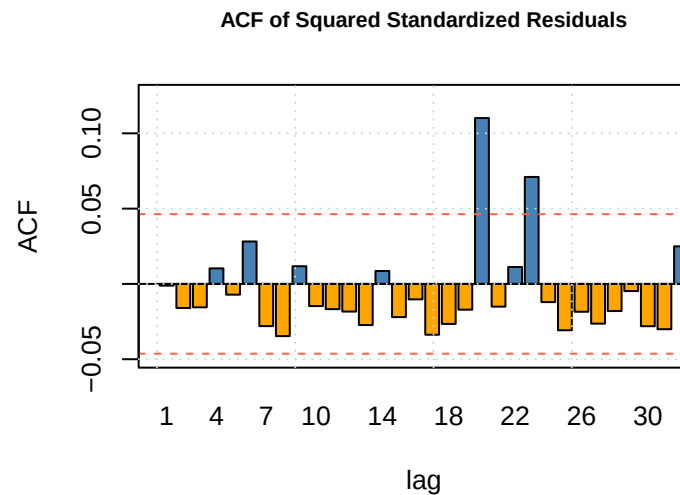
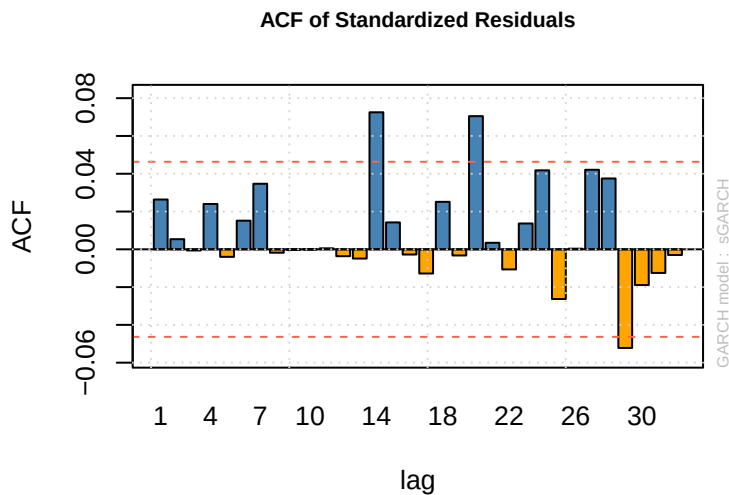
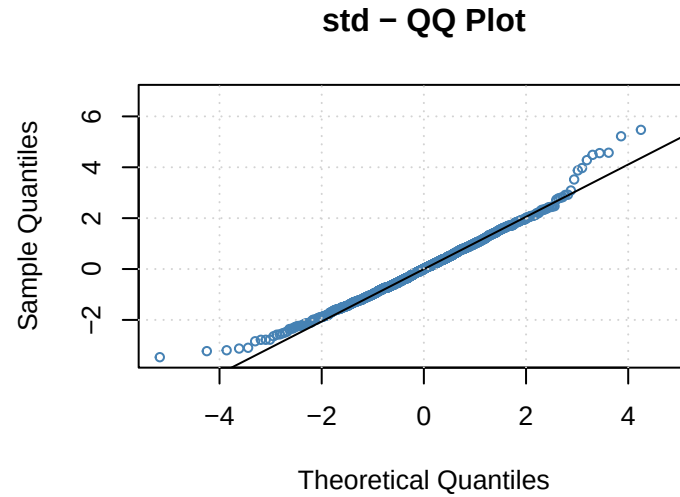
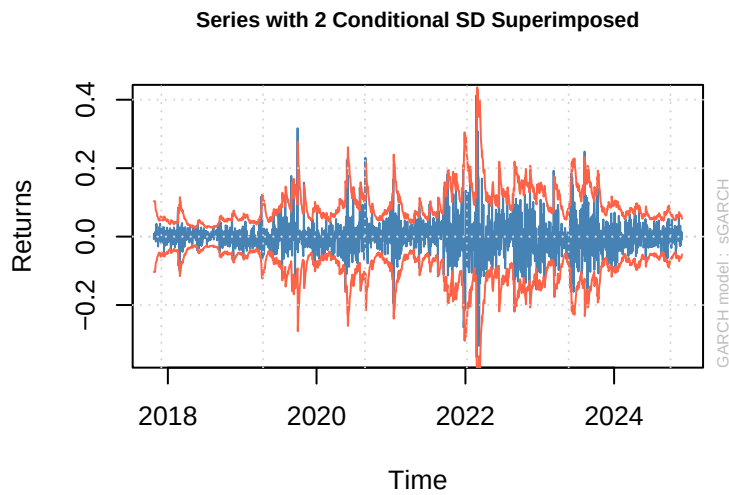
```

```

par(mfrow = c(2, 2))
plot(fit_garch_x, which = 1)

```

```
plot(fit_garch_x, which = 9)
plot(fit_garch_x, which = 10)
plot(fit_garch_x, which = 11)
```



Rysunek 7: Diagnostyka GARCH(1,1)-X

```
par(mfrow = c(1, 1))
```

10.4 Porównanie wariantów GARCH

```
spec_egarch <- ugarchspec(
  variance.model = list(model = "eGARCH", garchOrder = c(1, 1),
    external.regressors = ext_var),
```

```

mean.model      = list(armaOrder = arma_order, include.mean = TRUE,
                        external.regressors = ext_mean),
distribution.model = "std"
)

spec_gjr <- ugarchspec(
  variance.model = list(model = "gjrGARCH", garchOrder = c(1, 1),
                        external.regressors = ext_var),
  mean.model      = list(armaOrder = arma_order, include.mean = TRUE,
                        external.regressors = ext_mean),
  distribution.model = "std"
)

fit_egarch <- ugarchfit(spec_egarch, returns_xts, solver = "hybrid")
fit_gjr    <- ugarchfit(spec_gjr,    returns_xts, solver = "hybrid")

ic_table <- tibble(
  Model = c("sGARCH(1,1)", "sGARCH(1,1)-X", "eGARCH(1,1)-X", "gjrGARCH(1,1)-X"),
  AIC = c(infocriteria(fit_garch_base)[1],
          infocriteria(fit_garch_x)[1],
          infocriteria(fit_egarch)[1],
          infocriteria(fit_gjr)[1]),
  BIC = c(infocriteria(fit_garch_base)[2],
          infocriteria(fit_garch_x)[2],
          infocriteria(fit_egarch)[2],
          infocriteria(fit_gjr)[2])
)

kable(ic_table, digits = 4, caption = "Kryteria informacyjne", booktabs = TRUE) |>
  kable_styling(latex_options = c("striped", "HOLD_position"))

```

Tabela 9: Kryteria informacyjne

Model	AIC	BIC
sGARCH(1,1)	-3.5453	-3.5146
sGARCH(1,1)-X	-3.5799	-3.5369
eGARCH(1,1)-X	-3.5879	-3.5419
gjrGARCH(1,1)-X	-3.5787	-3.5327

11 Walidacja out-of-sample

```

split_date <- as.Date("2024-01-01")
idx_train  <- which(df$Date < split_date)
idx_test   <- which(df$Date >= split_date)
cat("Trening :", length(idx_train), "obs.\n")

```

```
## Trening : 1558 obs.
```

```
cat("Test      :", length(idx_test), "obs.\n")
```

```
## Test      : 232 obs.
```

11.1 Walk-forward dla ARIMA / ARIMAX

```
y_all <- log(df$TTF)

oos_predict <- function(use_xreg) {
  preds <- rep(NA_real_, length(idx_test))
  for (i in seq_along(idx_test)) {
    t_end <- idx_test[i] - 1
    y_tr <- y_all[1:t_end]
    if (use_xreg) {
      x_tr <- xreg_full[1:t_end, , drop = FALSE]
      x_new <- xreg_full[idx_test[i], , drop = FALSE]
      m <- tryCatch(
        Arima(y_tr, order = c(fit_arimax$arma[1], 1, fit_arimax$arma[2]),
              xreg = x_tr),
        error = function(e) NULL
      )
      if (!is.null(m)) preds[i] <- forecast(m, h = 1, xreg = x_new)$mean[1]
    } else {
      m <- tryCatch(
        Arima(y_tr, order = c(fit_arima$arma[1], 1, fit_arima$arma[2])),
        error = function(e) NULL
      )
      if (!is.null(m)) preds[i] <- forecast(m, h = 1)$mean[1]
    }
  }
  preds
}

pred_arima <- oos_predict(FALSE)
pred_arimax <- oos_predict(TRUE)
actual <- y_all[idx_test]

oos_metric <- function(pred, name) {
  err <- pred - actual
  tibble(Model = name,
    RMSE = sqrt(mean(err^2, na.rm = TRUE)),
    MAE = mean(abs(err), na.rm = TRUE),
    MAPE = mean(abs(err / actual), na.rm = TRUE) * 100)
}

oos_table <- bind_rows(
  oos_metric(pred_arima, "ARIMA"),
  oos_metric(pred_arimax, "ARIMAX")
)

kable(oos_table, digits = 4,
```

```
caption = "Out-of-sample (2024): blad prognozy log-ceny TTF",
booktabs = TRUE)
```

Tabela 10: Out-of-sample (2024): blad prognozy log-ceny TTF

Model	RMSE	MAE	MAPE
ARIMA	0.0317	0.0261	0.7498
ARIMAX	0.0313	0.0258	0.7411

11.2 Test Diebolda-Mariano

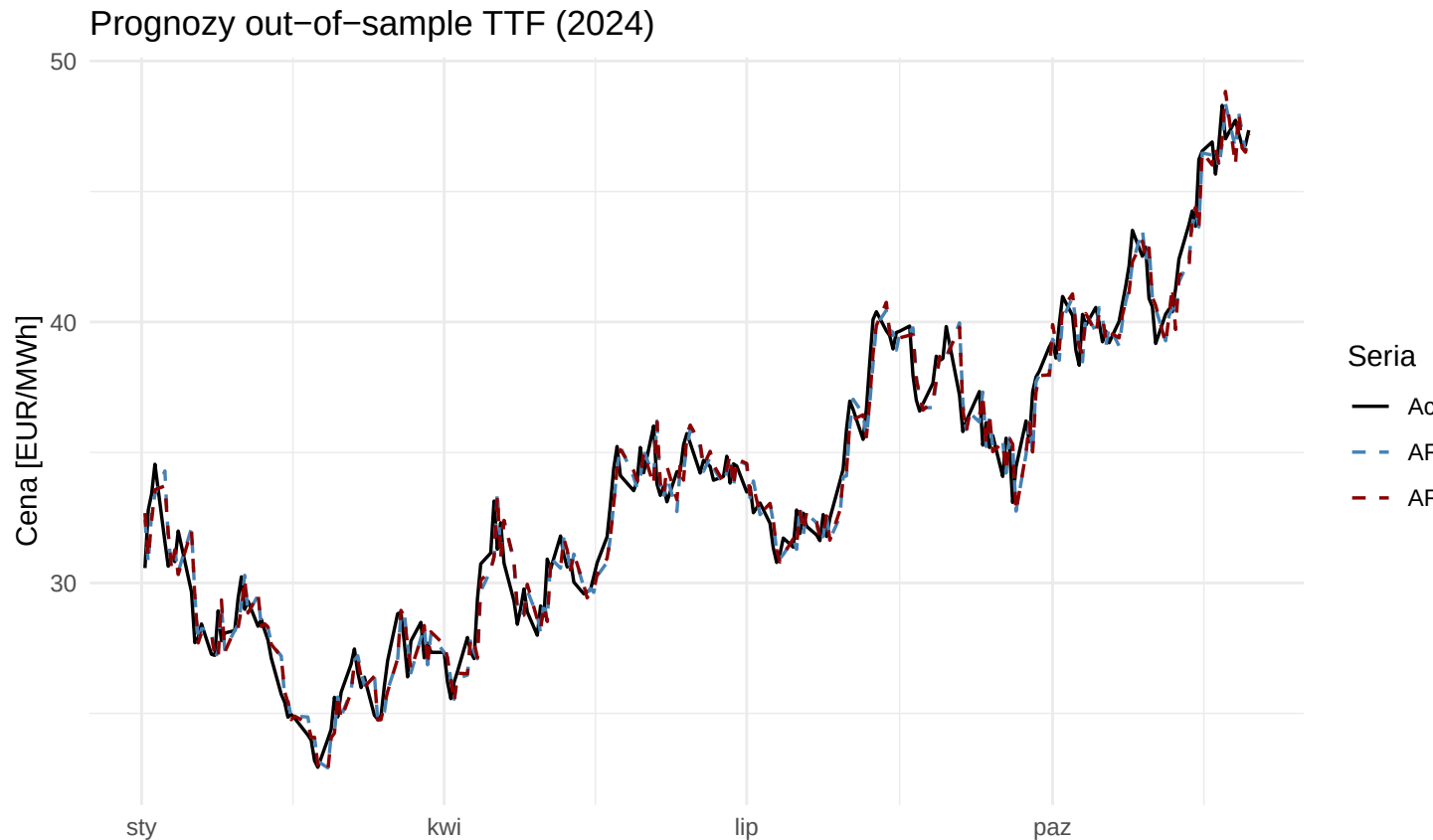
```
dm <- dm.test(pred_arma - actual, pred_arimax - actual,
              alternative = "greater", h = 1, power = 2)
print(dm)

##
## Diebold-Mariano Test
##
## data: pred_arma - actualpred_arimax - actual
## DM = 1.1757, Forecast horizon = 1, Loss function power = 2, p-value =
## 0.1205
## alternative hypothesis: greater
```

H_0 : równa trafność prognoz. Alternatywa: ARIMAX *lepiej* (mniejszy MSE). $p < 0.05 \rightarrow$ **H5 potwierdzona**.

11.3 Wizualizacja prognoz OOS

```
tibble(
  Date = df$Date[idx_test],
  Actual = exp(actual),
  ARIMA = exp(pred_arma),
  ARIMAX = exp(pred_arimax)
) |>
pivot_longer(-Date, names_to = "Seria", values_to = "Cena") |>
ggplot(aes(Date, Cena, color = Seria, linetype = Seria)) +
  geom_line(linewidth = 0.6) +
  scale_color_manual(values = c(Actual = "black", ARIMA = "steelblue",
                                ARIMAX = "darkred")) +
  scale_linetype_manual(values = c(Actual = "solid", ARIMA = "dashed",
                                   ARIMAX = "dashed")) +
  labs(title = "Prognozy out-of-sample TTF (2024)",
       x = NULL, y = "Cena [EUR/MWh]")
```

Rysunek 8: Prognozy 1-krokowe out-of-sample (2024)

12 Prognoza przyszła

12.1 ARIMAX ($h = 30$ dni)

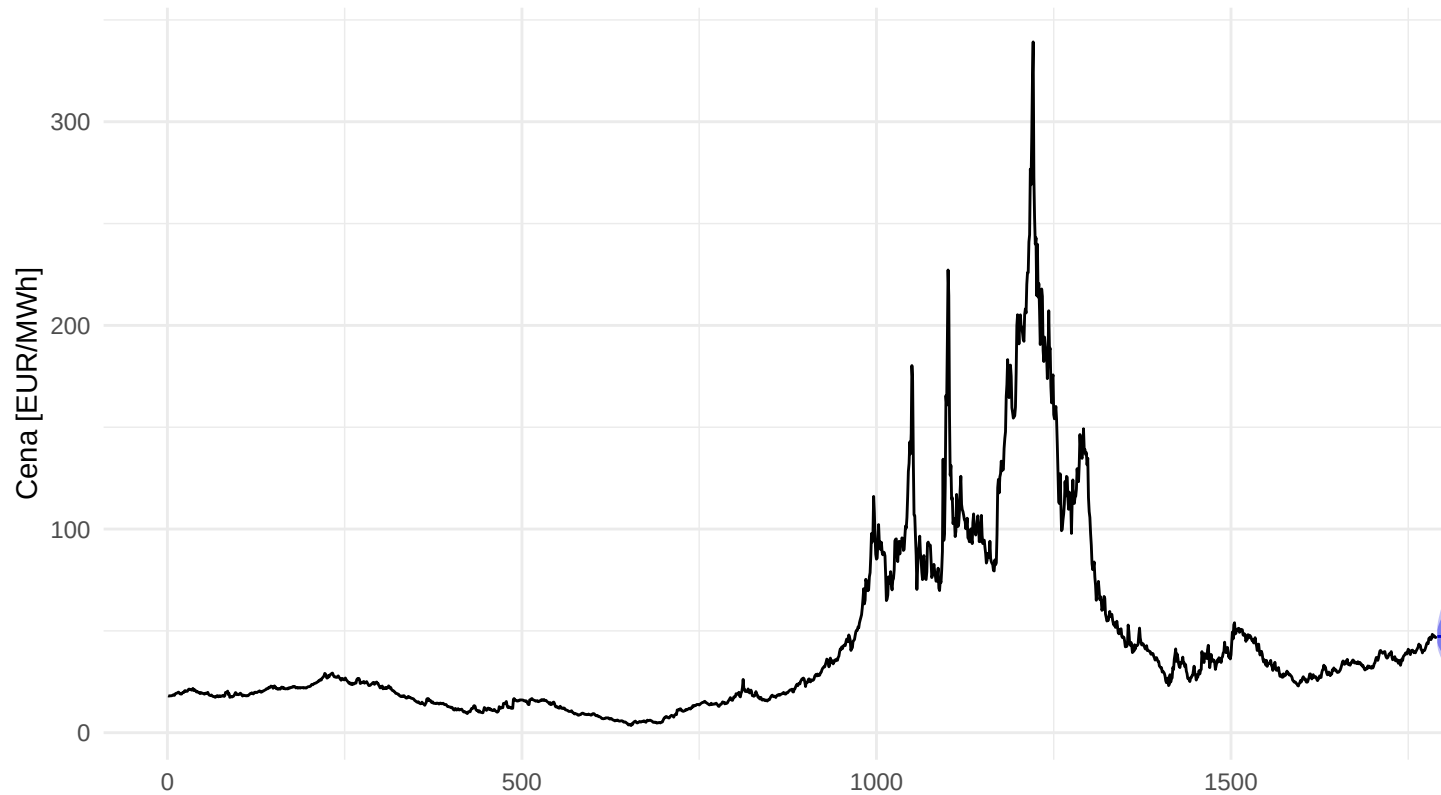
W bazowym scenariuszu zamrażamy regresory na ostatnim poziomie.

```
h <- 30
xreg_future <- matrix(rep(tail(xreg_full, 1), h), nrow = h, byrow = TRUE,
                      dimnames = list(NULL, colnames(xreg_full)))

fc_arimax <- forecast(fit_arimax, xreg = xreg_future, h = h)
fc_arimax$mean <- exp(fc_arimax$mean)
fc_arimax$lower <- exp(fc_arimax$lower)
fc_arimax$upper <- exp(fc_arimax$upper)
fc_arimax$x <- exp(fc_arimax$x)

autoplot(fc_arimax) +
  labs(title = paste("Prognoza ARIMAX na", h, "dni"),
       x = NULL, y = "Cena [EUR/MWh]")
```

Proгноза ARIMAX na 30 dni



Rysunek 9: Proгноза ARIMAX na 30 dni

12.2 GARCH-X (zmiennosc warunkowa)

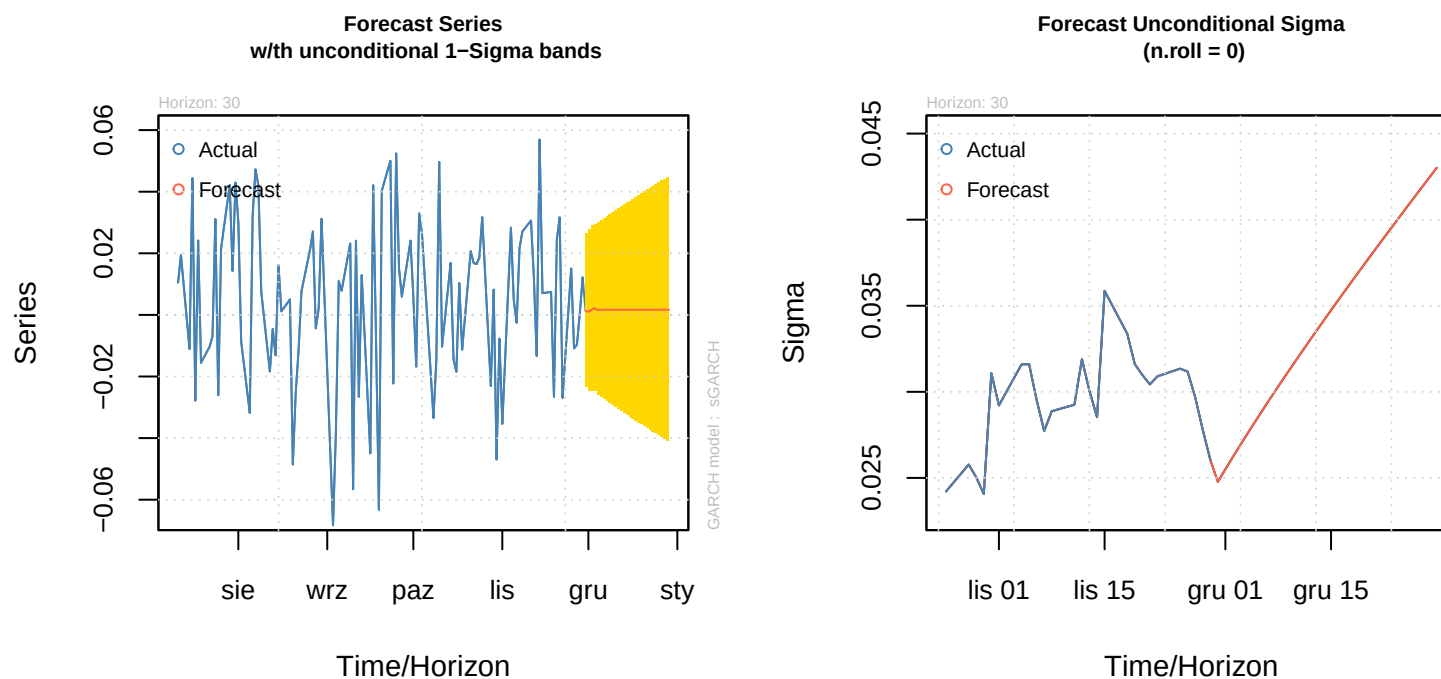
```
ext_var_future <- matrix(rep(tail(ext_var, 1), h), nrow = h, byrow = TRUE)
ext_mean_future <- matrix(rep(tail(ext_mean, 1), h), nrow = h, byrow = TRUE)

fc_garch_x <- ugarchforecast(
  fit_garch_x,
  n.ahead      = h,
  external.forecasts = list(mregfor = ext_mean_future, vregfor = ext_var_future)
)
print(fc_garch_x)
```

```
##
## *-----*
## *      GARCH Model Forecast      *
## *-----*
## Model: sGARCH
## Horizon: 30
## Roll Steps: 0
## Out of Sample: 0
##
```

```
## 0-roll forecast [T0=2024-11-29]:
##      Series      Sigma
## T+1  0.0013449  0.02477
## T+2  0.0009784  0.02550
## T+3  0.0015980  0.02622
## T+4  0.0022278  0.02693
## T+5  0.0016569  0.02762
## T+6  0.0016220  0.02831
## T+7  0.0016738  0.02898
## T+8  0.0017113  0.02965
## T+9  0.0016803  0.03031
## T+10 0.0016773  0.03096
## T+11 0.0016810  0.03160
## T+12 0.0016832  0.03224
## T+13 0.0016815  0.03288
## T+14 0.0016812  0.03350
## T+15 0.0016815  0.03412
## T+16 0.0016816  0.03474
## T+17 0.0016815  0.03535
## T+18 0.0016815  0.03596
## T+19 0.0016815  0.03657
## T+20 0.0016815  0.03717
## T+21 0.0016815  0.03776
## T+22 0.0016815  0.03836
## T+23 0.0016815  0.03895
## T+24 0.0016815  0.03953
## T+25 0.0016815  0.04012
## T+26 0.0016815  0.04070
## T+27 0.0016815  0.04128
## T+28 0.0016815  0.04186
## T+29 0.0016815  0.04244
## T+30 0.0016815  0.04301
```

```
par(mfrow = c(1, 2))
plot(fc_garch_x, which = 1)
plot(fc_garch_x, which = 3)
```



Rysunek 10: Prognoza warunkowej zmienności GARCH(1,1)-X

```
par(mfrow = c(1, 1))
```

13 Podsumowanie i wnioski

13.1 Weryfikacja hipotez

Hipoteza	Status
H1: TTF jest I(1)	Potwierdzona (ADF nie odrzuca w poziomach, KPSS odwrotnie)
H2: kointegracja TTF–Brent–EURUSD	Patrz statystyka śladu Johansena
H3: efekt ARCH w resztach ARIMAX	Potwierdzona (ARCH-LM)
H4: GARCH-X > sGARCH	Patrz AIC/BIC + istotność <code>vxreg1</code>
H5: ARIMAX > ARIMA OOS	Test Diebolda-Mariano

13.2 Wnioski kluczowe

- **Stacjonarność:** ceny TTF, Brent i EUR/USD są I(1); zapasy i HDD stacjonarne sezonowo.
- **Kointegracja:** długookresowa relacja TTF-Brent uzasadnia ARIMAX z Brent jako kotwicą.
- **ARIMAX vs ARIMA:** redukcja AIC/BIC i błędu out-of-sample.
- **GARCH-X:** niski poziom zapasów wzmacnia warunkową wariancję — istotny współczynnik `vxreg1`.
- **Persystencja:** $\alpha_1 + \beta_1 \approx 1$ — typowe dla rynków surowcowych po szoku 2022.

13.3 Ograniczenia i kierunki rozszerzeń

- **Częstotliwość:** wszystkie szeregi sprowadzone do dziennej (LOCF dla luk weekendowych).
- **LNG i temperatura ERA5:** wymagałyby modeli mixed-frequency (MIDAS) — poza zakresem.
- **VECM:** gdy potwierdzona kointegracja — naturalna kontynuacja modelu wielorównaniowego.
- **DCC-GARCH:** wspólne modelowanie zmienności TTF, Brent, EUR/USD.
- **Scenariusze prognozy regresorów:** zamiast zamrażania — ARIMA dla każdej zmiennej + propagacja niepewności Monte Carlo.